



M256 Unit 4

UNDERGRADUATE COMPUTING

Software development with Java



More conceptual
models

Unit **4**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenq@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008

Copyright © 2007, 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5477 3



CONTENTS

1	Introduction	5
1.1	The aims of this unit	6
1.2	Studying this unit	6
2	Developing a conceptual model for the Shopping System	7
2.1	Identifying classes and attributes	7
2.2	Identifying associations	16
2.3	Identifying invariants	23
2.4	Derived attributes and associations	29
2.5	Alternative conceptual models for the Shopping System	31
3	Developing a conceptual model for the Training System	35
4	Summary	40
	Glossary	41
	Index	42

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In *Unit 3* we introduced you to the components of a *conceptual model*, that is, a model of the real-world area within which a system is to operate (the *system domain*), and to some of the techniques involved in producing such a model from the requirements document for the system. You learnt how to identify a set of conceptual classes, representing categories of real-world entities, and how a class diagram can represent these classes and the relationships between them.

SAQ 1

Along with a class diagram, what else should be included in a conceptual model for a proposed software system?

ANSWER.....

The model should also include text consisting of class descriptions and statements of any required invariants.

You also learnt that the conceptual model generates the *initial structural model* for the system. What this means is that the initial structure for the software system is based on the conceptual model: that is, the system will have software classes corresponding to the conceptual classes, with attributes, associations and invariants in accordance with the conceptual model. The initial structural model is the first of a series of models of the system that will lead to the eventual implementation of the system in code.

SAQ 2

- (a) In what ways are the conceptual model and the initial structural model for a system the same? In what ways are they different?
- (b) Explain what is meant by the words 'structural' and 'initial' in the context of the initial structural model.

ANSWER.....

- (a) The conceptual model and the initial structural model look the same – they involve the same diagram and text. They are different in their perspectives: the conceptual model represents the real world, whereas the initial structural model represents software.
- (b) The model is 'structural' because it describes only the structure (the static, unchanging aspects) of the proposed system: the classes, their attributes and the ways in which the classes are related to each other. It does not describe dynamic aspects of the system – the ways in which objects of the classes will interact.
The model is 'initial' because it is only the first of a number of similar models that will be produced during the development of the proposed system.

1.1 The aims of this unit

This unit aims to:

- ▶ give you the opportunity to practise and consolidate what you have learnt in *Unit 3* (Sections 2 and 3);
- ▶ extend your understanding of the concepts and techniques covered in that unit (Sections 2 and 3).

You will be considering two examples: the Shopping System, whose requirements document you met in *Unit 2*, and another small system, the Training System.

1.2 Studying this unit

This unit is shorter than *Unit 3*, and it introduces few new concepts. You may find that it takes less than the time indicated on the *Study Calendar* to complete.

There is no computer-based work in this unit, but it contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. Some of these are quite substantial. They form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

We estimate that about three-quarters of your time will be spent on Section 2, in which a conceptual model for the Shopping System is constructed. Section 3 involves you in producing a complete conceptual model for another small system.

You will need to refer to the *Handbook* during your study of this unit.

2

Developing a conceptual model for the Shopping System

You were introduced to the Shopping System in *Unit 2*. Starting from the requirements document produced in that unit, the aim of this section is to produce a conceptual model for the Shopping System. In so doing, we will apply (and expand upon, where appropriate) the approaches discussed in *Unit 3*.

Section 6 of the *Handbook* contains the requirements document for the Shopping System, and you will need to refer to it throughout this section.

2.1 Identifying classes and attributes

Identifying classes

On the basis of the *System domain* and *Use cases* sections of the Shopping System requirements document, we will identify significant categories of real-world entities which give rise to candidate classes for inclusion in the conceptual model. We begin by conducting a textual analysis to identify nouns and noun phrases, choosing one representative term for each group of synonyms.

Recall that synonyms refer to entities of the same kind, such as 'product' and 'computing product'.

Exercise 1

Below, with synonyms removed, is our list of nouns and noun phrases from the *System domain* section of the Shopping System requirements document (see Section 6 of the *Handbook*), in the order in which they appear in that document. Read this section, confirming that each term listed (or its equivalent) is mentioned.

company
product
website
time
a number of registered customers
customer
product details
order
system
session
item (i.e. an item in an order, which we refer to from now on as an 'order item')
product A
product B

Discussion.....

You should have seen that each of the terms (or a synonym) is mentioned.

Although the idea of identifying nouns and noun phrases is simple in principle, in practice you may need to think carefully to decide whether particular terms refer to the same, or to different entities, and different decisions can be possible depending on the interpretation of the requirements. This is a difficulty of working only with the requirements document, without a client who can clarify matters. Two aspects of this exercise might have caused you particular thought.

- 1 In our list, we distinguished 'product' and 'order item' as different concepts.
 - ▶ 'Product' corresponds to the specification of a *type* of product stocked by the company. For example, a product may be specified with description 'OCS Laser Printer, Model 2007', product code 'P213' and a price per item of £199. 'Product' is conceptually similar to 'film' in the DVD Library System discussed in *Unit 3*. It provides information common to a number of items, in the way that information about a film was common to a number of DVDs.
 - ▶ 'Order item' corresponds to a single entry in an order form, for example, an order entry for one printer of the type with description 'OCS Laser Printer, Model 2007'. Though there must presumably be physical items of a product (perhaps in a warehouse) corresponding to its stock level, an order item does not correspond directly to a particular physical item until the order is packed and shipped: our system is not concerned with these activities.
- 2 We identified a single kind of entity, 'order', relating to orders. You might have included two: 'submitted order' and 'unsubmitted order', perhaps. The modelling of orders in this system is an interesting point, with several different possibilities, and we continue discussion of it in Subsection 2.5. Until then, we will continue to model orders as one kind of entity.

It should be evident by now that in practice it is usually impossible to make decisions about the conceptual model for even a simple system by following a set of standard techniques in a predefined order. Although we introduce and use a set of techniques and guidelines in a roughly linear manner, you should not feel hidebound by this approach. Sometimes it will be useful to employ the techniques roughly in the order presented, sometimes less so. In fact, as we suggested in *Unit 3*, with practice you may gain confidence in deciding on various aspects of a model in parallel, referring to the different techniques and guidelines as appropriate. For example, in the process outlined above of trying to identify classes, you might already be thinking about identifying associations between these classes (customers *have* orders, for example). What is vital is to combine common sense with judicious use of the techniques to deliver a conceptual model that is an accurate reflection of the system domain.

That said, you will now continue to identify candidate classes for the Shopping System conceptual model.

Exercise 2

Produce a list of distinct entities mentioned in the *Use cases* section of the Shopping System requirements document, grouping together synonymous nouns and noun phrases. Do not include those already listed in Exercise 1.

Discussion.....

Here is our list. You may have used synonyms for some items; allowing for this, your list should correspond to the one below.

customer registration	information about the session	product price
customer name	product code	stock level of product
customer address	product description	information about the order
customer password	order item price	start time of session
customer number	time of ordering	staff member
customer's details	order number	use case D
entered password	date of order	
appropriate message	total cost of order	

In forming your list for Exercise 2 you might have debated the necessity to include both 'order item price' and 'product price'. After all, we have not included both 'order item description' and 'product description', because we have assumed that this information about an order item is determined by the related product.

However, the requirements mention the possibility that after adding an item to an order, the price of the product might change, and that in such circumstances the order item price should be unaffected. A customer who had decided to purchase an item at a certain listed price would not take kindly to being invoiced for that item at a higher price, and presumably the company would prefer not to discount items that a customer had agreed to buy when they cost more. So each item in an order needs to record the price when the customer added that item to their order. The object diagram in Figure 1 illustrates a situation where a product's price differs from that of one of its order items: `orderItem3` was ordered when `product36`'s price was 6.50.

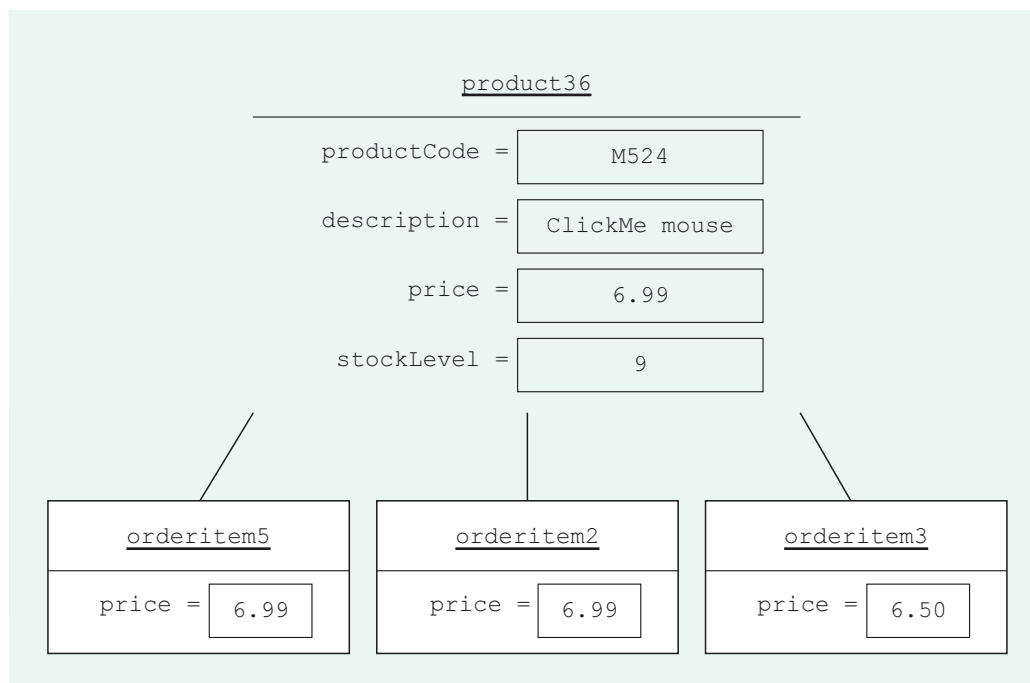


Figure 1 does not show the orders to which order items are related.

Figure 1 An order item's price may differ from the corresponding product's price

The entities listed in Exercises 1 and 2 give rise to the candidate classes at this stage. We now use the rejection guidelines introduced in *Unit 3* to investigate whether any entities can be discarded as candidate classes. Here, for reference are these guidelines.

Is it a property?

Is it simply a property of some other type of entity in the system domain, or does it contribute to the system domain in its own right? If the former, it is likely to be appropriately modelled by an attribute of a particular class rather than a class itself. An example from the Shopping System is 'name', a property of a customer.

Does it refer to a behaviour of the system?

Does it name some process or result of a process that the system has to carry out? The entity 'product details' is an example of this kind.

Is it outside the scope of the system?

Is it a word or phrase that is used to help describe the general purpose of the system or what it consists of, but that does not actually refer to significant entities in the system domain? An example of this is 'website'.

Does it refer to an aspect of the user interface?

Does it concern the interaction of a user with the system, rather than something that is part of the system itself? 'Entered password' is of this kind, since it concerns the interaction of a user with the system, and not something that is part of the system itself.

Does it refer to connections between other significant entities in the system domain?

Such an entity might be more appropriately modelled as an association between two classes rather than as a class itself. There are no such examples in the system domain here.

Is it a word or phrase that is used in talking about systems in general, rather than a reference to something in the system domain?

The entity 'system' is one such example.

Is it simply part of a language idiom?

That is, is it a figure of speech, not contributing in itself to the description of the system domain? An example of this in the requirements document for the Shopping System is 'time' ('... may only be involved in a single session at any one time').

These guidelines are by no means definitive. There can be circumstances in which a guideline suggests that a particular entity might be rejected for modelling by a class, but where other factors will mean that you want to retain it. Equally, an entity may be rejected for reasons not covered by the guidelines.

Neither do entities always fit neatly into unique categories. There can be different valid reasons for rejecting entities. For example, we might categorise 'product details' as referring to a behaviour of the system, or as part of a language idiom – shorthand for product description, price etc. When narrowing down the list of entities, it is vital to bear in mind that the aim is for each class to represent a real-world category of things that have specific significance for the system domain. When deciding whether to reject an entity, you need to take all the available information into account.

Below are the terms from Exercise 1. Alongside each, we have indicated our decision to reject or retain the entity as giving rise to a candidate class. In the case of rejection, a reason (which may be one of several possible) is given.

company (reject: outside the scope of the system)
 product (retain)
 website (reject: outside the scope of the system)
 time (reject: language idiom)
 a number of registered customers (reject: language idiom)
 customer (retain)
 product details (reject: refers to the system's behaviour)
 order (retain)
 system (reject: talks about systems in general)
 session (retain)
 order item (retain)
 product A (reject: language idiom – referring to a particular product)
 product B (reject: ditto)

In the following exercise, you will analyse the remaining entities for the Shopping System.

Exercise 3

For each item in the list given in the solution to Exercise 2, decide whether to reject it as giving rise to a candidate class, using the rejection guidelines where appropriate. For each entity, state either that you have retained or rejected it and give a reason in the case of rejection.

Discussion.....

Below are our decisions. Your reasons may be different, but you should have decided to reject the same entities.

customer registration (retain)
 customer name (reject: attribute of customer)
 customer address (reject: attribute of customer)
 customer password (reject: attribute of customer)
 customer number (reject: attribute of customer)
 customer's details (reject: refers to the system's behaviour)
 entered password (reject: aspect of user interface)
 appropriate message (reject: aspect of user interface)
 information about the session (reject: refers to the system's behaviour)
 product code (reject: attribute of product)
 product description (reject: attribute of product)
 order item price (reject: attribute of order item)
 time of ordering (reject: language idiom)
 order number (reject: attribute of order)
 date of order (reject: attribute of order)
 total cost of order (reject: attribute of order)
 product price (reject: attribute of product)
 stock level of product (reject: attribute of product)
 information about the order (reject: refers to the system's behaviour)
 start time of session (reject: attribute of session)
 staff member (reject: outside the scope of the system)
 use case D (reject: language idiom)

It might initially appear inconsistent to reject 'staff member' as being outside the scope of the system, whilst 'customer' has been retained. Both are used in similar contexts, for example:

The staff member provides the system with ...

The customer provides the system with ...

If the latter were the only kind of reference to 'customer', we would reject 'customer' as being outside the scope of the system. But 'customer' has another role; customers' details are required, and customers are connected with sessions and with orders. So customers are significant entities in the system domain as well as users of the system, and as such, the system needs to hold information about them.

Here is the list of entities that now potentially give rise to classes:

customer registration, session, customer, order, order item, product.

To complement the analysis so far, we can attempt to validate our choices by considering the following aspects of a system domain that may appropriately be modelled as classes:

These categories were described more fully in Subsection 3.2 of *Unit 3*.

- ▶ tangible (i.e. physical) entities;
- ▶ roles (that people and things play);
- ▶ events (happenings or incidents);
- ▶ organisational units (structures to which people or things belong);
- ▶ abstract entities (things that exist but which, like the play Hamlet, are not tangible).

Exercise 4

Consider each of the entities above. For each, decide whether it fits into one of the five categories, and if so which one.

Do not spend too much time on this: not all of them are easy to categorise.

Discussion.....

Our list is:

customer registration: event
 customer: role
 session: event
 order: abstract entity
 order item: abstract entity
 product: abstract entity

The first three ('customer registration', 'customer' and 'session') are straightforward, the others require more discussion.

'Order' could perhaps be alternatively classified as an organisational unit (to which order items belong).

'Order item' could perhaps be classified as a tangible entity, because for each order item there must be at least one particular physical item in stock (though, as already mentioned, the correspondence between an order item and a particular physical item is not a concern of this system).

'Product', as we discussed earlier, provides information common to a number of items, so might be classified as an organisational unit to which order items belong.

Two kinds of event: 'customer registration' and 'session' are modelled by candidate classes for the Shopping System at this stage. In *Unit 3*, you learnt that events sometimes, but not always, merit modelling as classes.

SAQ 3

Unit 3 gave three conditions under which it might be appropriate to model an event as a class. State these.

ANSWER.....

- 1 When the event has properties of its own.
- 2 When the event involves more than two other significant entities.
- 3 When different occurrences of the event involving the same entities need to be recorded.

Exercise 5

For each of the events 'customer registration' and 'session', explain, with reference to SAQ 3, whether it should be retained or rejected.

Discussion.....

The event 'customer registration' satisfies none of the criteria given in SAQ 3 for modelling as a class. It has no properties of its own, and it involves only a single customer, who registers just once.

A session has a property of its own, start time, and therefore 'session' merits modelling as a class.

The candidate classes for the Shopping System are now the following:

session, customer, order, order item, product.

Working from a requirements document

Above, we considered identifying categories of likely classes (modelling tangible entities, roles etc.) as a validation process, having already carried out a textual analysis and applied the rejection guidelines. Such consideration can give confidence that the right sorts of thing have been identified.

The next exercise explores how a simple textual analysis may not identify every candidate class.

Exercise 6

The following extract is taken from a requirements document for a system for managing holiday accommodation.

The company has several branches, each of which manages holiday villas situated in the geographical area for which the branch is responsible. Customers of the company can book to stay in villas. When a customer books a villa, a reference number is allocated and recorded, along with the period the customer has booked for.

- (a) Carry out a textual analysis to identify candidate classes for a conceptual model for this system.
- (b) Reread this extract from the requirements document. Identify an additional class, modelling an event, that is needed for the conceptual model. Explain your reasoning.

Discussion.....

(a) Textual analysis gives the following list of candidate classes.

company
branch
villa
geographical area
customer
reference number
period

(b) The event of booking is of central significance to the system domain described.

That, coupled with the need to record a reference number and period when a villa is booked, should lead you to deduce that a booking class is necessary: the booking of a villa by a customer is an event about which there is a need to record information, yet the extract from the requirements document contained no noun or noun phrase which referred to a booking. Had the requirements document included the phrase 'can make a booking' instead of 'can book', the identification of the need for a 'booking' class would have been much more straightforward.

The holiday system example which formed the basis of Exercise 6 demonstrates that even minor differences in the wording of the requirements document can affect the ease with which candidate classes can be identified. This illustrates the importance of not relying on textual analysis alone. When you are creating a conceptual model, as well as using textual analysis, you should also consider whether other classes are needed, using the five common categories as a framework. In doing this, though, you should limit your considerations to the information in the requirements document; in other words, you should be aiming to identify significant entities that are implicit in the description, as well as those that arise from textual analysis. For example, based on the description of the holiday system given in Exercise 6, we would not include a class 'customer account', even though common sense tells us that this could be a significant aspect of the client's business. In practice such issues would be discussed with the client and, if agreed, built into the requirements document. In M256 you have to work from the information provided, and will need to base your decisions on the information given, so if something is not mentioned or implied in the requirements document, you should conclude that it is not required.

Identifying attributes

As a by-product of listing possible classes for the Shopping System, we have also identified a number of attributes for those classes. For example, a product has a code, a description and so on. We can thus begin the process of writing the class descriptions that will form part of the conceptual model. Here are the classes that we have identified.

Session, Customer, Order, OrderItem, Product.

The class descriptions for the Session and Customer classes, incorporating the attributes that have been identified so far, are as follows.

Class Session A customer session on the company website

Attributes

startTime The time the session started

Class	Customer	A customer of the company
Attributes		
	customerNumber	The unique number of the customer
	name	The name of the customer
	address	The address of the customer
	password	The password of the customer

However, some attributes may not be explicitly identified when listing classes: the requirements document must be consulted again for attributes that have not yet materialised. Here, the requirements document distinguishes between submitted and unsubmitted orders, which can be modelled by assigning an attribute, which we will call `status`, to the `Order` class.

Other possibilities for modelling these different kinds of order are discussed in Subsection 2.5.

Exercise 7

Write down class descriptions for `Order`, `OrderItem` and `Product`, including all the attributes now identified for those classes.

Discussion.....

Class	Order	A customer's order
Attributes		
	orderNumber	The unique number of the order
	totalCost	The total cost of the order
	date	The date of the order
	status	The status (submitted or unsubmitted) of the order

Class	OrderItem	An item in a customer's order
Attributes		
	price	The price of the item at the time of selection

Class	Product	A product stocked by the company
Attributes		
	code	The unique code of the product
	description	The description of the product
	price	The current price of the product
	stockLevel	The current stock level of the product

It is important to note that it is not appropriate to include as attributes information that will be recorded using associations. For example, each order item is connected to a particular product. In the conceptual model, such connections are modelled by links of associations – as you will see, we will associate `OrderItem` with `Product`. It is not appropriate, in the conceptual model, to represent connections between order items and products using an attribute, e.g. `product` or `code` in the class `OrderItem`.

You will learn that in a Java system both attributes and links can be implemented using instance variables. However, they must be distinguished at this stage.

It is worth emphasising that the list of classes we have at this point is by no means finalised. In a real software development project the process of creating a conceptual model would be iterative, with all aspects of the emerging model regularly under review.

2.2 Identifying associations

A textual analysis of the requirements document, to find the grammatical structures that connect the entities being modelled as classes, can help in identifying candidate associations. As with classes, however, the identification of associations is not an automatic process.

SAQ 4

The *Add item* use case (G) begins: 'The customer identifies a product.' Why is it inappropriate to include an association, say *identifies*, between *Customer* and *Product*?

ANSWER.....

We noted previously that 'customer' appears in two different guises in the requirements document for the system – as a user of the system and as a significant entity in the system domain.

This part of use case G treats a customer as a user of the system: identification of a product is part of how the user interacts with the system. 'Identifies' therefore describes an aspect of the user interface, and so should not be modelled as part of the system domain.

Exercise 8

Conduct a textual analysis of the requirements document for the Shopping System, and identify candidate associations. Do not at this stage consider the relationship between different products – i.e. that one may be an alternative for another – as we will consider that later.

Discussion.....

Here are the candidate associations we identified. We have included one reference in support of each association; a different occurrence in the requirements document may suggest a slightly different name for the association.

isFor between *Customer* and *Session* ('... a current session for that customer' – described in use case B, *Start session*).

has between *Customer* and *Order* ('... the customer ... their order' – described in the *System domain* section).

isIn between *OrderItem* and *Order* ('... each item in the order' – described in use case D, *List submitted orders*).

isOf between *OrderItem* and *Product* ('... an item of the product' – described in use case G, *Add item*).

Multiplicities

Before considering multiplicities for the Shopping System associations, we will revise some important facts about multiplicities in general.

Recall that, given an association between classes A and B, the multiplicity of the association at the B end describes, for an object of class A, how many objects of class B it may be linked with at any one time via that association. Each association therefore has two multiplicities – one at either end. Determining this information involves analysing the corresponding connections between real-world entities.

A multiplicity has one of the following forms.

- ▶ m (where m is a positive integer), e.g. 1.
- ▶ $m..n$ (where m is 0 or a positive integer, and n is an integer larger than m). This notation refers to the range of integers from m to n , inclusive. $0..1$ is a common example of this.
- ▶ $m..*$ (where m is 0 or a positive integer). This notation refers to the range of integers from m upwards, inclusive of m . Common examples of this are $0..*$ and $1..*$.

When you are assigning multiplicities, it is particularly important that you distinguish between cases where there must always be at least one link and cases where it is possible that no links may exist. The most common decisions you will have to make are:

- ▶ whether a multiplicity should be 1 or $0..1$;
- ▶ whether a multiplicity should be $0..*$ or $1..*$.

SAQ 5

Why is it not sensible to have a multiplicity of 0? (It might help if you sketch a diagram involving two classes joined by an association with a multiplicity of 0 at one end.)

ANSWER.....

Suppose that the multiplicity of an association between classes A and B at the B end is 0, as shown in Figure 2. This would mean that each object of class A is always linked via the association to zero objects of class B. That is, an object of class A can never be linked via the association to an object of class B – there can never be any links! Hence there is no need for an association between A and B.



Figure 2 Fragment of a class diagram showing a multiplicity of 0

While a multiplicity of 0 is not possible, a multiplicity of a range *involving* 0, for example, $0..1$ or $0..*$, is very common. Consider Figure 3, which is an extract from a class diagram for a university system. A student has either a single counsellor or none. A counsellor may counsel any number of students, including none.



Figure 3 The association counsels between Counsellor and Student

Continuing with this example of the University System, the object diagrams in Figure 4 show an example of two Counsellor objects and four Student objects.

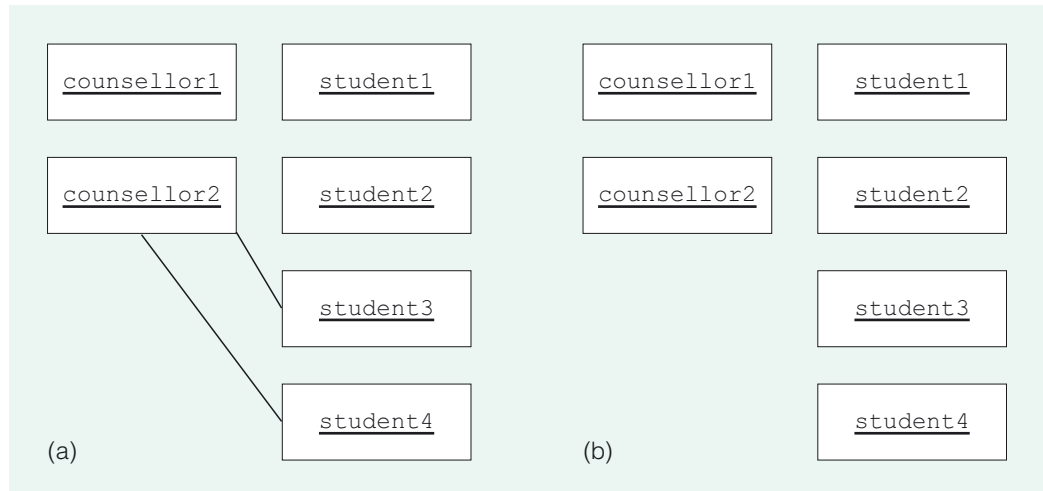


Figure 4 Two object diagrams for the University System

At the point in time represented in Figure 4(a), there are some links between Counsellor objects and Student objects. However, counsellor1 is not linked to any Student object, a situation compatible with the multiplicity of counsels at the Student end including 0. Similarly, the fact that neither student1 nor student2 is linked to a Counsellor object is compatible with the multiplicity of counsels at the Counsellor end including 0.

In fact, it is compatible with the multiplicities that at some time *no* Counsellor object is linked to *any* Student object. This possibility is illustrated in Figure 4(b).

SAQ 8

Suppose that the multiplicity at the Counsellor end of counsels is changed to 1, reflecting the fact that each student must now have a counsellor. Why could neither of the situations represented in the object diagrams in Figure 4 then occur?

ANSWER.....

If the multiplicity of counsels at the Counsellor end is 1, then each Student object must always be linked to a Counsellor object.

The situation in Figure 4(a) cannot then occur because neither student1 nor student2 is linked to a Counsellor object. The situation in Figure 4(b) cannot then occur because *none* of the Student objects is linked to a Counsellor object.

In Exercise 8, we identified the following associations for the conceptual model for the Shopping System.

- isFor between Session and Customer
- has between Customer and Order
- isIn between OrderItem and Order
- isOf between OrderItem and Product

Exercise 9

Draw a class diagram showing the classes and associations above, and determine and include the multiplicities for each association.

Discussion.....

Our class diagram is shown in Figure 5.

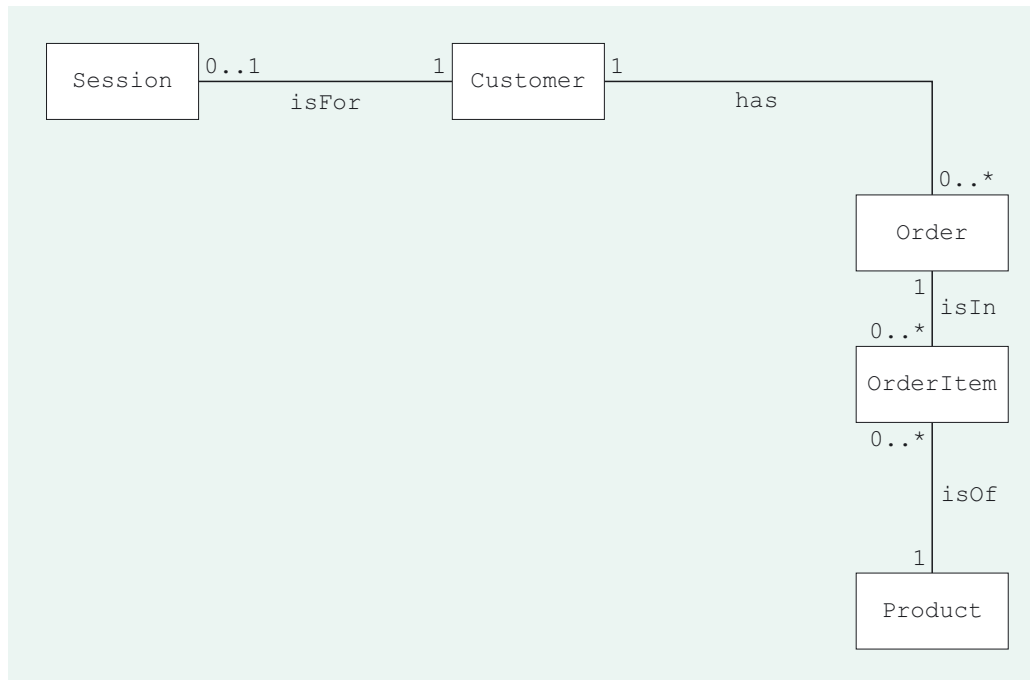


Figure 5 A class diagram for the Shopping System

Here is our rationale for the multiplicities.

- ▶ **isFor.** There is no requirement to retain information about sessions that are complete. Consequently, only current sessions exist, of which any given customer may have at most one. The multiplicity of **isFor** at the **Session** end is therefore 0..1. Each session relates to a single customer, so the multiplicity of **isFor** at the **Customer** end is 1.
- ▶ **has.** A customer may have zero or more orders (although no more than one may be unsubmitted), so the multiplicity of **has** at the **Order** end is 0..*. Each order belongs to a single customer, so the multiplicity of **has** at the **Customer** end is 1.
- ▶ **isIn.** Because a customer has an empty order before adding any items, an **Order** object may be linked to no **OrderItem** objects. There is no upper limit on the number of items in an order; so the multiplicity of **isIn** at the **OrderItem** end is 0..*. An **OrderItem** object is part of just one order, so the multiplicity of **isIn** at the **Order** end is 1.
- ▶ **isOf.** An order item is of a single product, so the multiplicity of **isOf** at the **Product** end is 1. There may be zero or more order items of a product, depending on how many have been ordered by customers, so the multiplicity of **isOf** at the **OrderItem** end is 0..*.

Recursive associations

One aspect of the Shopping System conceptual model that we have yet to consider is the fact that a product may be considered as an alternative to another product. This means that there is a relationship between objects of the same class, `Product`. All the associations you have seen so far have been between two distinct classes but, as here with `Product`, an association may exist between a class and itself, in which case it is called a **recursive association**. In a class diagram, a recursive association is simply shown as going from the class in question to itself, as shown in Figure 6.

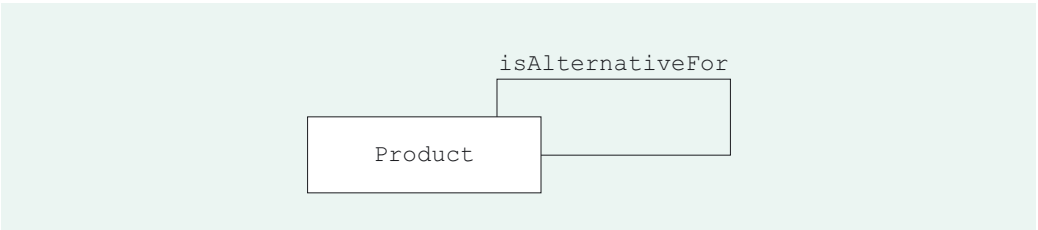


Figure 6 A recursive association involving the `Product` class

An example of links of this particular recursive association involving the `Product` class, which we have called `isAlternativeFor`, is shown in the object diagram in Figure 7. Each link represents the fact that the products represented by the objects at either end of the link are considered to be suitable alternatives to each other.

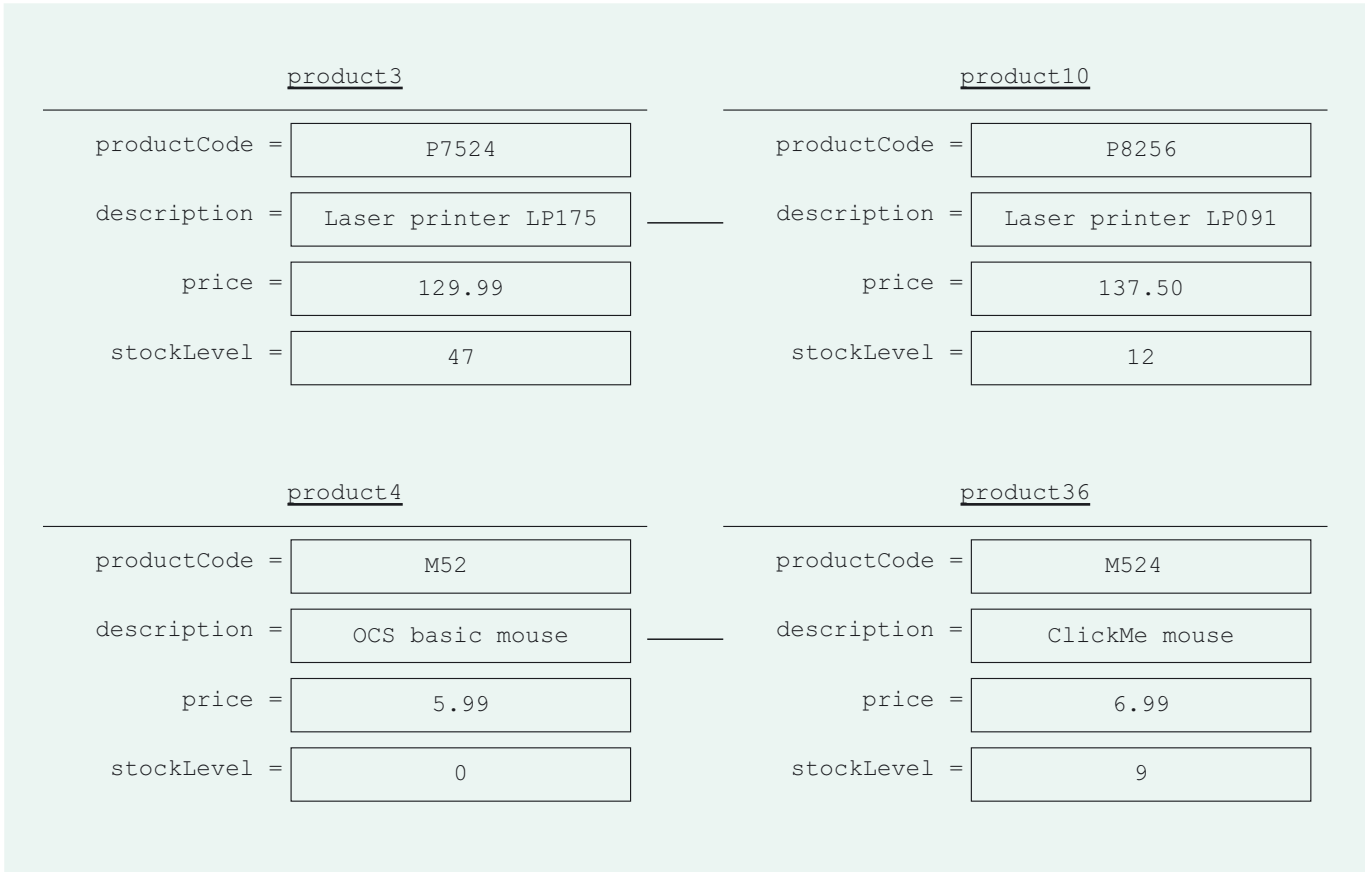


Figure 7 Two links of `isAlternativeFor`

According to the requirements document for the Shopping System, in the case of some, but not all, products, another product exists that may be a suitable alternative, and where there is an alternative, the two products are paired, so that each is an alternative for the other. The multiplicity at each end of `isAlternativeFor` is therefore `0..1`, as illustrated in Figure 8.

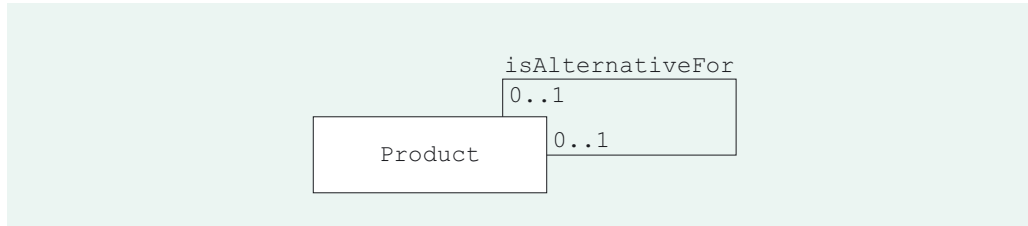


Figure 8 `isAlternativeFor` with multiplicities

Recursive associations can also be used to model a situation where one object of a class may be related to several other objects of the same class. To illustrate this, imagine that the Hospital System discussed in *Unit 3* has some additional requirements that involve nurses. These might lead to a `Nurse` class in the conceptual model. Suppose that among these requirements was the following.

Some nurses supervise other nurses. A nurse is supervised by at most one other nurse.

A typical situation is shown in the object diagram in Figure 8. Each link in this diagram represents the fact that the nurse represented by the object on the left of the link supervises the nurse represented by the object on the right. Thus the nurse represented by the object `nurse1` supervises the nurses represented by `nurse2` and `nurse3`, `nurse2` supervises `nurse4` and `nurse5`, and so on.

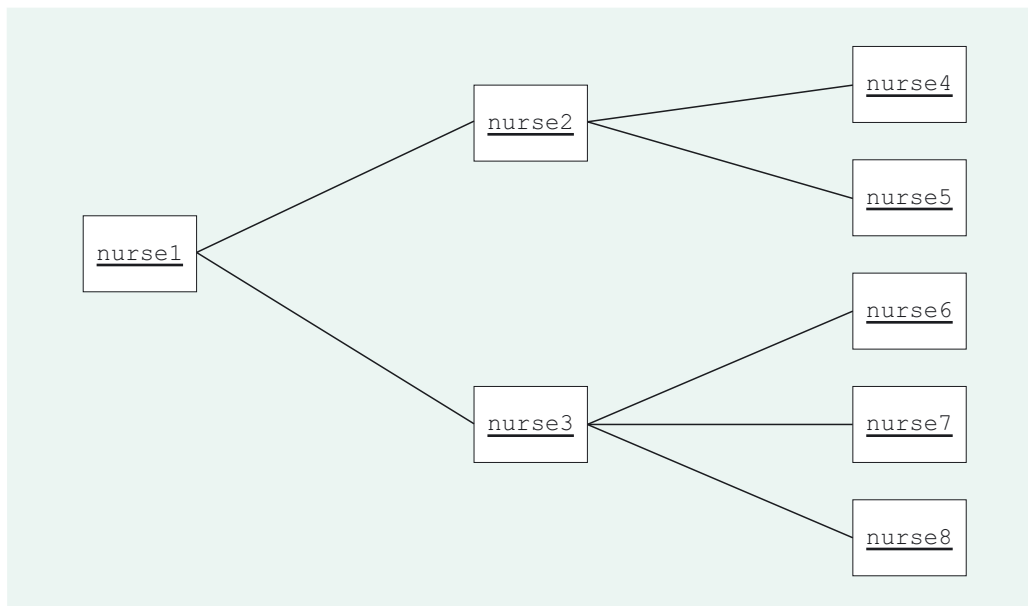


Figure 9 Nurses supervising other nurses

The links shown in Figure 9 are instances of an association, `supervises`, which is recursive. In determining the multiplicities of `supervises`, it is necessary to distinguish the supervisor's end from the supervisee's end. For example, in Figure 9, `nurse2` represents the supervisor of the supervisee who is represented by `nurse4`.

SAQ 6

- (a) What is the multiplicity of the `supervises` association at the supervisor's end?
 (b) What is the multiplicity of the `supervises` association at the supervisee's end?

ANSWER.....

- (a) The multiplicity is `0..1`. A nurse is supervised by one supervisor or none.
 (b) The multiplicity is `0..*`. A nurse can supervise any number of supervisees (including zero).

The `supervises` association can be depicted in a class diagram as shown in Figure 10. The directional arrow is needed to indicate the relationship between the roles at either end and the multiplicities. It makes it clear that a nurse can supervise many nurses but can be supervised by at most one nurse.

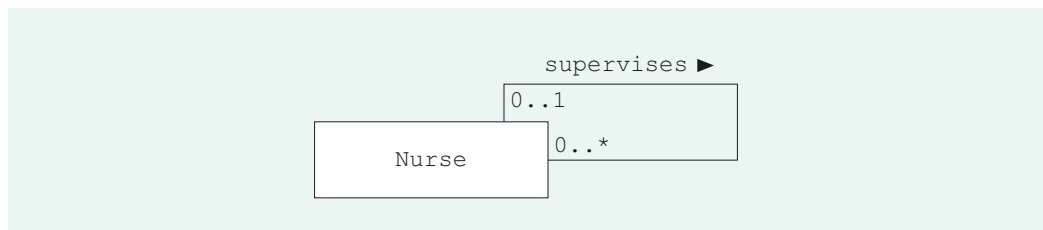


Figure 10 A recursive association involving the `Nurse` class

Below is a class diagram for the conceptual model of the Shopping System, now including the association `isAlternativeFor`. There is no need for a directional arrow in the case of `isAlternativeFor`, because the objects at either end of an `isAlternativeFor` link play exactly the same role, unlike `supervises`, where the different roles were supervisor and supervisee.

At this stage in forming a conceptual model for the Hospital System in *Unit 3*, we considered whether it was appropriate to model any generalisation relationships. It should be clear to you that this is not appropriate for any of the classes we are currently working with.

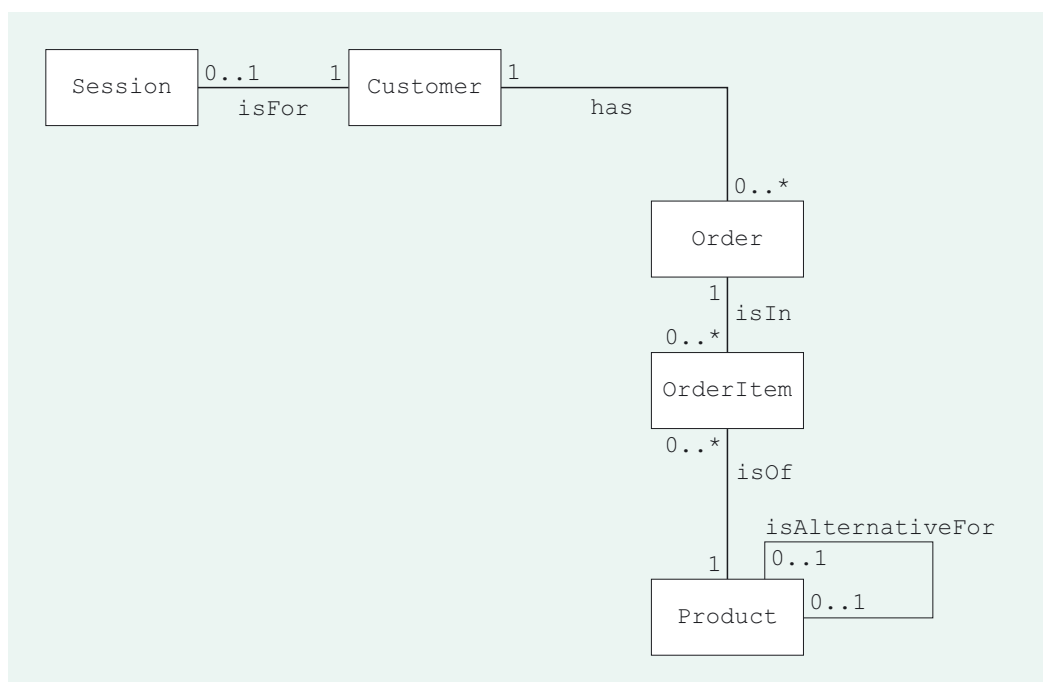


Figure 11 A class diagram for the Shopping System

2.3 Identifying invariants

In *Unit 3* you learnt that an invariant is a condition on the elements of the conceptual model that must always hold, and which follows from a real-world constraint. Some invariants, such as the values that a particular attribute may take, are straightforward and might be stated simply in a comment accompanying the relevant class description. Others are more complex, involving combinations of objects, attribute values and links.

Figure 11 above and the class descriptions below make up the conceptual model for the Shopping System so far.

Class	Session	A customer session on the company website
	Attributes	
	startTime	The time the session started
Class	Customer	A customer of the company
	Attributes	
	customerNumber	The unique number of the customer
	name	The name of the customer
	address	The address of the customer
	password	The password of the customer
Class	Order	A customer's order
	Attributes	
	orderNumber	The unique number of the order
	totalCost	The total cost of the order
	date	The date of the order
	status	The status (submitted or unsubmitted) of the order
Class	OrderItem	An item in a customer's order
	Attributes	
	price	The price of the item at the time of selection
Class	Product	A product stocked by the company
	Attributes	
	code	The unique code of the product
	description	The description of the product
	price	The current price of the product
	stockLevel	The current stock level of the product

To the textual part of this model we need to add any invariants not already evident from the class diagram and descriptions.

Exercise 10

- (a) Write down one invariant that is evident from the class diagram.
- (b) Write down one invariant that is evident from the class descriptions.

Discussion.....

Below we give one example for each part. In each case, there are several other similar examples you might have chosen.

- (a) Every `Session` object is linked to exactly one `Customer` object.
This invariant is apparent from the multiplicity of `isFor` at the `Customer` end being 1.
 - (b) No two `Customer` objects may have the same value for `customerNumber`.
This invariant is apparent from the description of customer numbers as unique in the comment for the `Customer` attribute `customerNumber`.
-

With reference to the solution to Exercise 10, it is important to be aware that whenever the multiplicity of an association is restricted in some way (that is, is not `0..*`), or whenever an attribute value is constrained, this represents an invariant – even although it does not need to be stated explicitly. You may also recall from *Unit 3* that our convention is simply to assume, without stating, common-sense restrictions on attribute values, such as that the stock level of a product must be zero or more.

Other invariants need to be stated explicitly. Consider the following two real-world constraints.

- 1 A customer may have no more than one unsubmitted order.
- 2 If a customer has an unsubmitted order, then they must also have a session.

Relevant real-world constraints such as these might not be stated explicitly in the requirements document. There is not, for example, an explicit statement that a customer may have an unsubmitted order only if they also have a session. Instead, we can infer this fact from the general description of a customer's interaction with the system, which includes statements such as:

Once a customer has started an online session, and opts to create an order ...

In the conceptual model, constraint 1 gives rise to the following invariant.

- 1 A `Customer` object may be linked to no more than one `Order` object whose `status` attribute has the value `unsubmitted`.

As you will see in Subsection 2.5, different conceptual models for the same system domain can require different invariants.

Exercise 11

State the invariant corresponding to constraint 2, above.

Discussion.....

- 2 If a `Customer` object is linked to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object is also linked to a `Session` object.

Exercise 12

Some further invariants are required for our Shopping System conceptual model.

- (a) One concerns a relationship between attribute values of the `Order` and `OrderItem` classes.
- (b) One concerns the number of `OrderItem` objects to which a submitted `Order` object must be linked.
- (c) One concerns which `Product` objects may *not* be linked via the `isAlternativeFor` association.

Write down real-world constraints that would lead to such invariants.

Discussion.....

The three constraints are:

- (a) The total cost of an order is the sum of the prices of all the items contained in that order.
 - (b) A submitted order must contain at least one order item.
 - (c) A product may not be an alternative for itself.
-

The first constraint listed in the solution to Exercise 12 corresponds to the following invariant (listed as invariant 3 in Section 7 of the *Handbook*).

For each `Order` object, its `totalCost` attribute value is equal to the sum of the `price` attribute values of all the `OrderItem` objects to which the `Order` object is linked.

SAQ 7, overleaf, asks you to consider what this invariant means in terms of objects and links of the conceptual model.

SAQ 7

For each of the diagrams in Figure 12, state whether the combination of objects and links portrayed satisfies the above invariant.

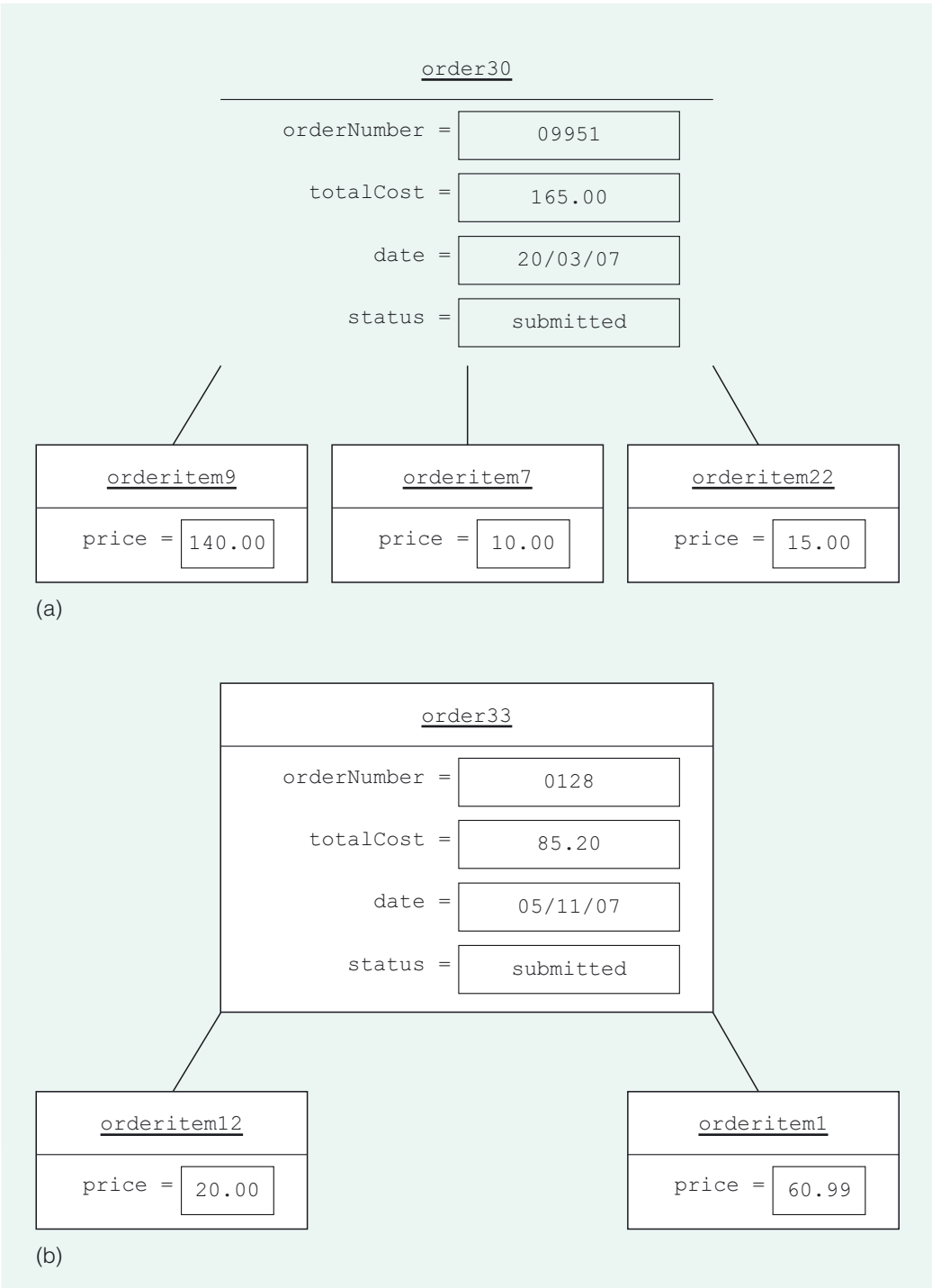


Figure 12 SubmittedOrder objects and their OrderItem objects

ANSWER.....

In Figure 12(a) the combination of objects and links satisfies invariant 3. The `totalCost` value of `order30` (165.00) is the sum of the `price` values of all the linked `OrderItem` objects (140.00, 10.00 and 15.00).

In Figure 12(b) the combination of objects and links does not satisfy invariant 3. The `totalCost` value of `order33` (85.20) is *not* the sum of the `price` values of all the linked `OrderItem` objects (20.00 and 60.99, whose sum is 80.99).

A consolidated list of invariants for the Shopping System conceptual model, including the invariants corresponding to the constraints in Exercise 12, is as follows.

- 1 A `Customer` object may be linked to no more than one `Order` object whose `status` attribute has the value `unsubmitted`.
- 2 If a `Customer` object is linked to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object is also linked to a `Session` object.
- 3 For each `Order` object, its `totalCost` attribute value is equal to the sum of the `price` attribute values of all the `OrderItem` objects to which the `Order` object is linked.
- 4 Each `Order` object whose `status` attribute has the value `submitted` is linked to at least one `OrderItem` object.
- 5 No `Product` object may be linked to itself.

Constraints that are not modelled by invariants

We have said that invariants are conditions on the conceptual model that are always true, and which correspond to real-world constraints. So you might have wondered why we did not include an invariant reflecting the following real-world constraint:

An item may not be added to an order if the stock level for the corresponding product is zero.

In attempting to formulate an invariant corresponding to this constraint, you might start by trying to state a restriction on a `Product` object with a `stockLevel` of 0. Perhaps such a `Product` object should not be linked to any `OrderItem` objects? But consider the following example. The stock level of a certain product is 1. A customer adds an item of that product to their order and the stock level of the product decreases to 0. The situation before and after the addition of the item is illustrated, overleaf, in Figure 13.

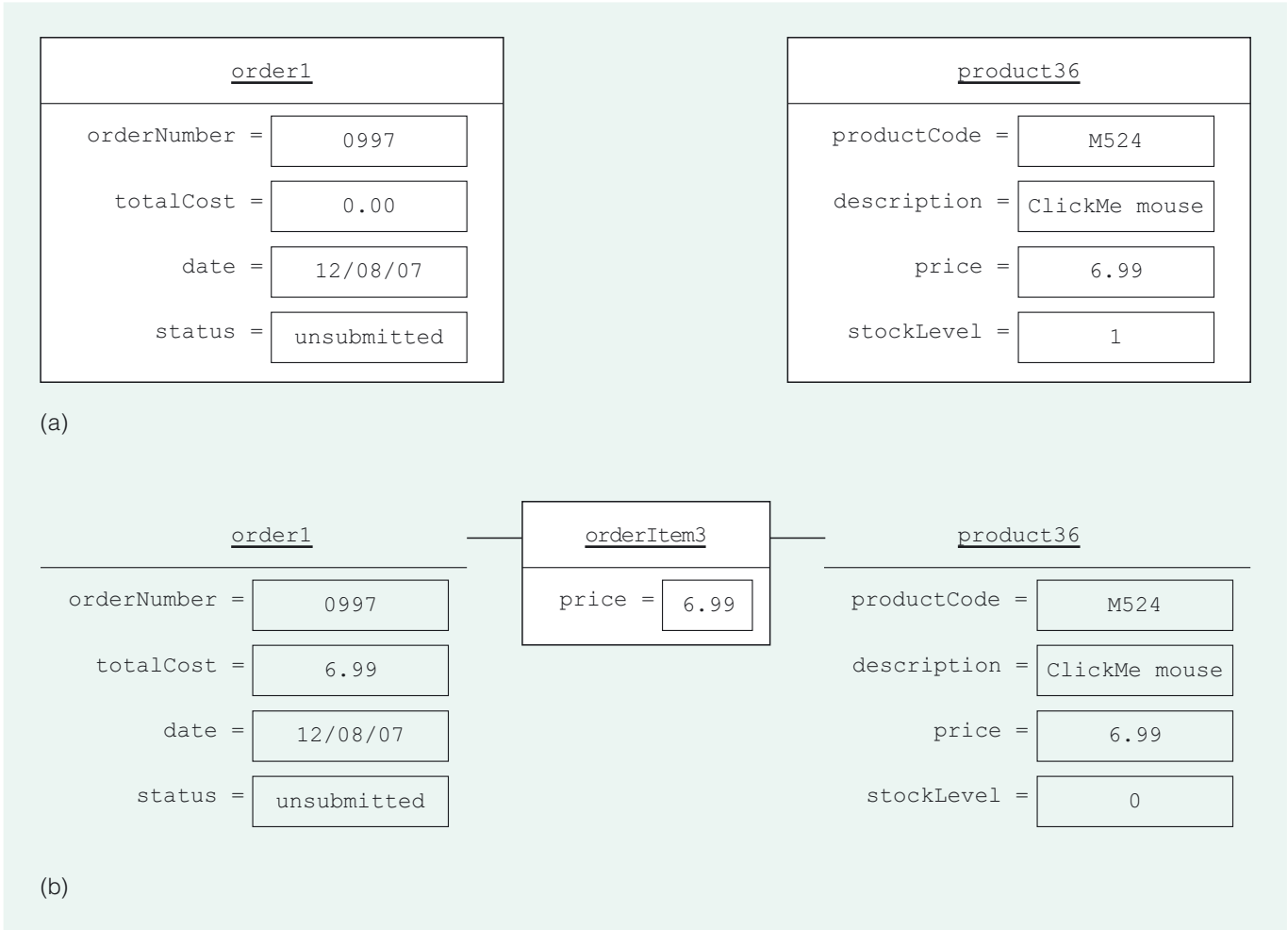


Figure 13 Object diagrams showing objects and links before and after an item has been added to an order

Before the addition of the item to the order, the stock level of the product corresponding to `product36` is 1, and `orderItem3` does not exist. Following successful selection of the item, `orderItem3` is linked to the relevant `Order` and `Product` objects, and `stockLevel` of `product36` is reduced to 0. The object `product36` remains linked to `orderItem3`, so clearly a `Product` object with `stockLevel` of 0 *can* be linked to `OrderItem` objects.

Trying to formulate an invariant corresponding to this constraint is doomed to failure. This is because the constraint specifies what *actions* are allowable when the system is in a particular state, rather than what *states* are allowable. In other words, it is a *dynamic* constraint rather than a *static* one. An invariant can only express a static constraint (by specifying what combinations of objects, attributes and links are allowable); it cannot express what can or cannot be done when the system is in a particular state. This particular constraint (that an item cannot be added to an order when the stock level is zero) must instead be addressed during dynamic modelling: it is a condition that must hold *prior* to a user adding an item to their order, rather than one that must *always* be true.

You will meet dynamic modelling in *Unit 6*.

2.4 Derived attributes and associations

In *Unit 3* we discussed the relationship between invariants and derived attributes and associations. At the conceptual modelling stage:

- an attribute of a class is described as derived if, for every object of that class, the object's attribute value can be deduced from other aspects of the model;
- an association is described as derived if all of its links can be deduced from other aspects of the model.

You learnt that the derivations involved were based on invariants. So once you have identified invariants, you need to explore whether they indicate that attributes or associations can be derived. Here are the invariants for the Shopping System again.

- 1 A `Customer` object may be linked to no more than one `Order` object whose `status` attribute has the value `unsubmitted`.
- 2 If a `Customer` object is linked to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object is also linked to a `Session` object.
- 3 For each `Order` object, its `totalCost` attribute value is equal to the sum of the `price` attribute values of all the `OrderItem` objects to which the `Order` object is linked.
- 4 Each `Order` object whose `status` attribute has the value `submitted` is linked to at least one `OrderItem` object.
- 5 No `Product` object may be linked to itself.

Exercise 13

By reviewing the invariants for the Shopping System, listed above, determine a derived attribute and explain how it is derived.

Discussion.....

The `totalCost` attribute of `Order` is derived.

The `totalCost` value for any `Order` object can be calculated by adding the `price` values of all the linked `OrderItem` objects. (This is illustrated by the example in Figure 12(a).)

The fact that the `totalCost` attribute of `Order` is derived is denoted by preceding its name with a slash, /, in the class description.

Exercise 14

Invariant 2 says that if a `Customer` object is linked (via the association `has`) to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object must also be linked (via the association `isFor`) to a `Session` object. Does this indicate that it is possible to derive the `isFor` association from the `has` association? Explain your answer.

Discussion.....

The association `isFor` cannot be derived from `has`, because not all `isFor` links can be deduced from `has` links. This is because a `Customer` object may be involved in an `isFor` link, but have no `has` links at all. That is (informally) it cannot be inferred which customers have sessions just by looking at which customers have unsubmitted orders, because a customer may have a session without having started an order.

There are no other derived attributes or associations in the Shopping System conceptual model. Here is the conceptual model we have now created.

Class diagram

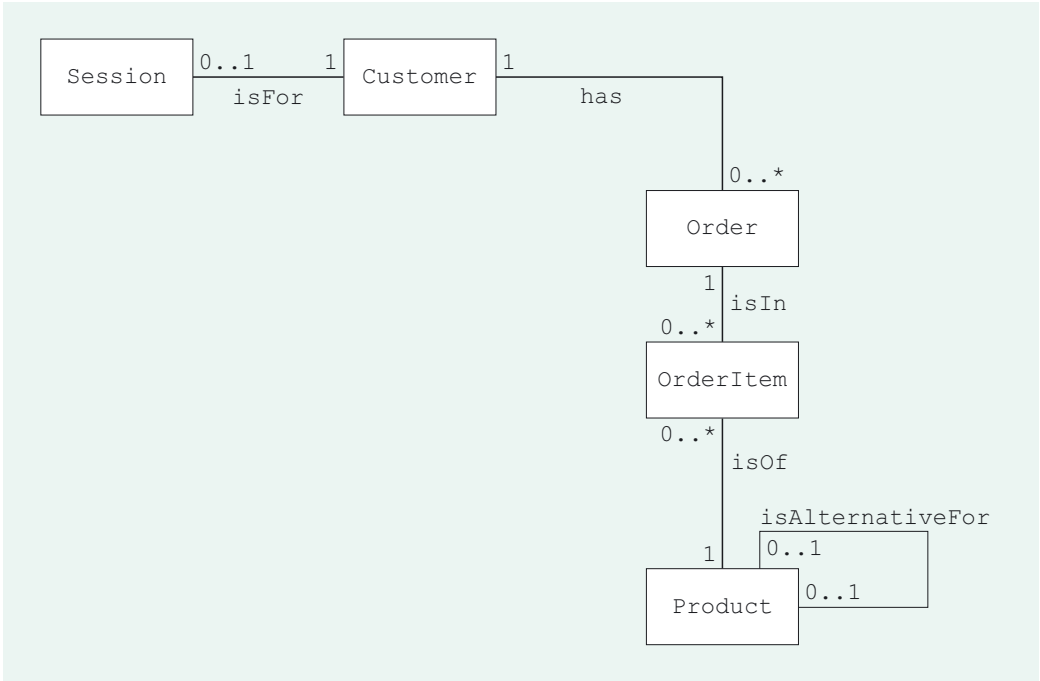


Figure 14 Class diagram for the Shopping System

Class descriptions

Class	Session	A customer session on the company website
Attributes		
	startTime	The time the session started
Class	Customer	A customer of the company
Attributes		
	customerNumber	The unique number of the customer
	name	The name of the customer
	address	The address of the customer
	password	The password of the customer
Class	Order	A customer's order
Attributes		
	orderNumber	The unique number of the order
	/totalCost	The total cost of the order
	date	The date of the order
	status	The status (submitted or unsubmitted) of the order

Class	OrderItem	An item in a customer's order
Attributes		
	price	The price of the item at the time of selection
Class	Product	A product stocked by the company
Attributes		
	code	The unique code of the product
	description	The description of the product
	price	The current price of the product
	stockLevel	The current stock level of the product

Invariants

- 1 A Customer object may be linked to no more than one Order object whose status attribute has the value `unsubmitted`.
- 2 If a Customer object is linked to an Order object whose status attribute has the value `unsubmitted`, then that Customer object is also linked to a Session object.
- 3 For each Order object, its `totalCost` attribute value is equal to the sum of the price attribute values of all the OrderItem objects to which the Order object is linked.
- 4 Each Order object whose status attribute has the value `submitted` is linked to at least one OrderItem object.
- 5 No Product object may be linked to itself.

2.5 Alternative conceptual models for the Shopping System

Whilst forming a conceptual model for the Shopping System, we have emphasised that there is no one correct model. In general, there may be several different ways to accurately represent a system domain. In the case of the Shopping System, instead of, as in our model, having a single `Order` class with an attribute `status` to distinguish between `unsubmitted` and `submitted` orders, you might have considered the following possibilities for modelling orders.

- 1 Separate classes `UnsubmittedOrder` and `SubmittedOrder`.
- 2 A single `Order` class as in our model, but with two associations with `Customer`: `hasSubmitted` and `hasUnsubmitted`.

The choice of a conceptual model is rarely clear-cut, and often no one model has advantages over others. In this course, we intend that you become proficient in producing a conceptual model that accurately represents the system domain; not that you become proficient in analysing different conceptual models. However, it is useful to consider the alternative possibilities listed above.

Alternative conceptual model 1

Though we have said that it is often not the case that one model is preferable to others, there is a strong argument that alternative 1 above is *not* a good representation of the real-world. Conceptually, whether an order is submitted or not, it is still the same order, i.e. the same entity. Therefore, a conceptual model that uses a single class to represent orders models the real-world situation more closely, and could therefore be considered a 'better' model.

Had we chosen this alternative conceptual model for the Shopping System, there would have been interesting repercussions on the subsequent software development. Recall that the conceptual model provides an initial structural model for the system. If we were continuing to develop the Shopping System based on this alternative conceptual model, we would initially consider the system as comprising software classes corresponding to the conceptual classes, which are related and constrained in accordance with the conceptual model. So we would start on the basis of software classes `Customer`, `UnsubmittedOrder`, `SubmittedOrder` and so on.

The next phase of development involves considering the dynamic aspects of the system, that is, determining how objects in the system will interact to enable the required tasks to be performed. This requires *additions* to the initial structural model. Classes will have methods assigned to them, for example. However, this phase may also prompt *changes* to the system's structure, as described below.

In the real world, when placing an order, an unsubmitted order *becomes* a submitted order: it is not the case that an unsubmitted order is destroyed and a new submitted order created to replace it. This is poorly reflected in a system with two classes, `UnsubmittedOrder` and `SubmittedOrder`, because an object of one class cannot be transformed into an object of another class. Consequently, when recording the submission of an order, it would be necessary to:

- ▶ create a new `SubmittedOrder` object;
- ▶ copy details into this object from the old `UnsubmittedOrder` object;
- ▶ link the new `SubmittedOrder` object to the `Customer` object;
- ▶ relink `OrderItem` objects from the old object to the new object.

Such an arrangement is possible, but is likely to be discarded at the dynamic modelling stage in favour of a single `Order` class, as in our original model, where the submission of an order would correspond simply to changing an `Order` object's `status` value.

Alternative conceptual model 2

In the case of alternative 2, there is little to choose between it and our original model. Figure 15 shows the class diagram for this alternative conceptual model.

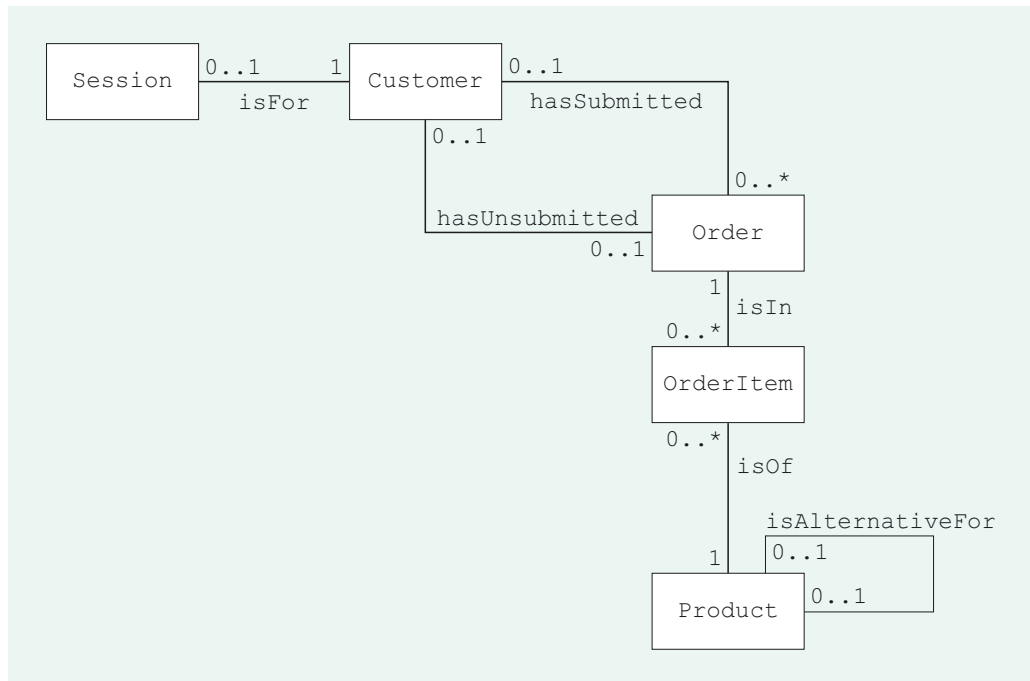


Figure 15 An alternative class diagram for the Shopping System

In this alternative model, the class `Order` has two associations with `Customer`: `hasUnsubmitted` and `hasSubmitted`. Whether an order is unsubmitted or submitted is represented by the existence of either a `hasUnsubmitted` link or a `hasSubmitted` link.

The description for the class `Order` in this alternative model is as follows.

Class	<code>Order</code>	A customer's order
Attributes		
	<code>orderNumber</code>	The unique number of the order
	<code>/totalCost</code>	The total cost of the order
	<code>date</code>	The date of the order

The above description differs from the one in our original model (see the listing after Figure 14) simply in not having an attribute `status`. Descriptions for the other classes in the alternative model would remain unchanged.

SAQ 8

Explain why the multiplicity of `hasSubmitted` at the `Customer` end is `0..1`.

ANSWER.....

An `Order` object represents an order, which must be either unsubmitted or submitted. If the former, then the `Order` object is not involved in a `hasSubmitted` link. If the latter, then the `Order` object is linked to a single `Customer` object via `hasSubmitted`. In other words, the `Order` object must be linked via `hasSubmitted` to either one `Customer` object or none.

Different invariants are required for this model than for our original. For example, the following is invariant 2 in the original model.

If a `Customer` object is linked to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object is also linked to a `Session` object.

In the alternative model, an invariant expressing the same constraint is required, but it has a different expression:

If a `Customer` object is linked via `hasUnsubmitted` to an `Order` object, then that `Customer` object must also be linked to a `Session` object.

Note the need to identify *which* link between the `Customer` object and the `Order` object is relevant – the invariant involves `hasUnsubmitted` links and not `hasSubmitted` links.

Exercise 15

(a) Consider each of the following invariants for our original model.

- 1 A `Customer` object may be linked to no more than one `Order` object whose `status` attribute has the value `unsubmitted`.
- 3 For each `Order` object, its `totalCost` attribute value is equal to the sum of the `price` attribute values of all the `OrderItem` objects to which the `Order` object is linked.
- 4 Each `Order` object whose `status` attribute has the value `submitted` is linked to at least one `OrderItem` object.
- 5 No `Product` object may be linked to itself.

For each decide whether a similar invariant (that is, one expressing the same real-world constraint) should be listed in the alternative model. If so, specify the invariant for the alternative model.

(b) State the additional invariant required in the alternative conceptual model to represent the fact that an order must be either `unsubmitted` or `submitted`, but not both.

Discussion.....

(a) Dealing with each invariant above in turn:

- 1 No similar invariant needs to be listed in the alternative model. In this model, the constraint that each customer may have no more than one unsubmitted order is represented by the multiplicity of `hasUnsubmitted` at the `Order` end being `0..1`.
- 3 This invariant is directly applicable to the alternative model. The same invariant should be listed unchanged.
- 4 This invariant represents a constraint which requires a different expression in the alternative model, as follows.

Each `Order` object which is linked via `hasSubmitted` to a `Customer` object is also linked to at least one `OrderItem` object.
- 5 This invariant is directly applicable to the alternative model. The same invariant should be listed unchanged.

(b) The additional invariant is as follows:

Each `Order` object is linked to a `Customer` object via either `hasUnsubmitted` or `hasSubmitted`, but not both.

In this section, you have been guided through the creation of a conceptual model for the Shopping System, applying and extending your understanding of the concepts and techniques introduced in *Unit 3*. You have also considered two alternative conceptual models for this system.

3 Developing a conceptual model for the Training System

In this final section, you will apply the approaches you have learnt about to the creation of a conceptual model for a company's training system. Here is the relevant part of a requirements document for this system.

Requirements document for the Training System

System domain

The company has several offices, each at a different address, and a number of employees. Each employee has a unique employee number, and works in a single office.

Each office has at least five trainees and one or two training managers working in it. The training managers in an office are jointly responsible for the training of all the trainees in that office. An office may have employees working in it who are neither trainees nor training managers.

The company's training department plans training sessions that trainees may attend. The department holds information only about future training sessions. As specialist equipment not owned by the company is required, each training session is held at an external institution. Details of all external institutions in which training sessions have been held are retained (for the purpose of planning further training sessions). In addition the company records whether an external institution was recommended to them by another institution on its list.

A training manager may make reservations for training sessions on behalf of trainees in their office. Each reservation is made by a single training manager, on behalf of just one trainee. There may be no more than one current reservation on behalf of each trainee. A training session may have many reservations, with each reservation being for a single training session.

Use cases

The system will allow an administrator in the training department to do the following:

- A List employees.** The administrator identifies an office. The system displays the name and employee number of each employee working in that office.
- B Give trainee's session details.** The administrator identifies a trainee. The system displays the title and date of any training session for which the trainee is reserved, along with the name and training management qualifications of the training manager who made the reservation.
- C Reserve training session.** The administrator identifies a training session, a trainee and a training manager. The system records that the training manager has reserved a place on the session for the trainee.
- D List external institutions.** For each external institution, the system displays the name and address of the institution, together with details of any specialist equipment available there and the name of the institution (if any) that recommended that institution. The titles and dates of all training sessions planned for that institution (including those training sessions for which there are currently no reservations) are also displayed.
- E Identify training session with fewest reservations.** The system displays details (title and date) of the training session(s) with the fewest reservations.

Exercise 16

Produce a conceptual model for the Training System. As suggested previously in this unit, you may want to identify different aspects of your model in parallel, applying the techniques and guidelines you have learnt about as and when these are helpful.

Discussion.....

Here is our conceptual model. We comment afterwards on aspects of it. Other models are possible.

Class diagram

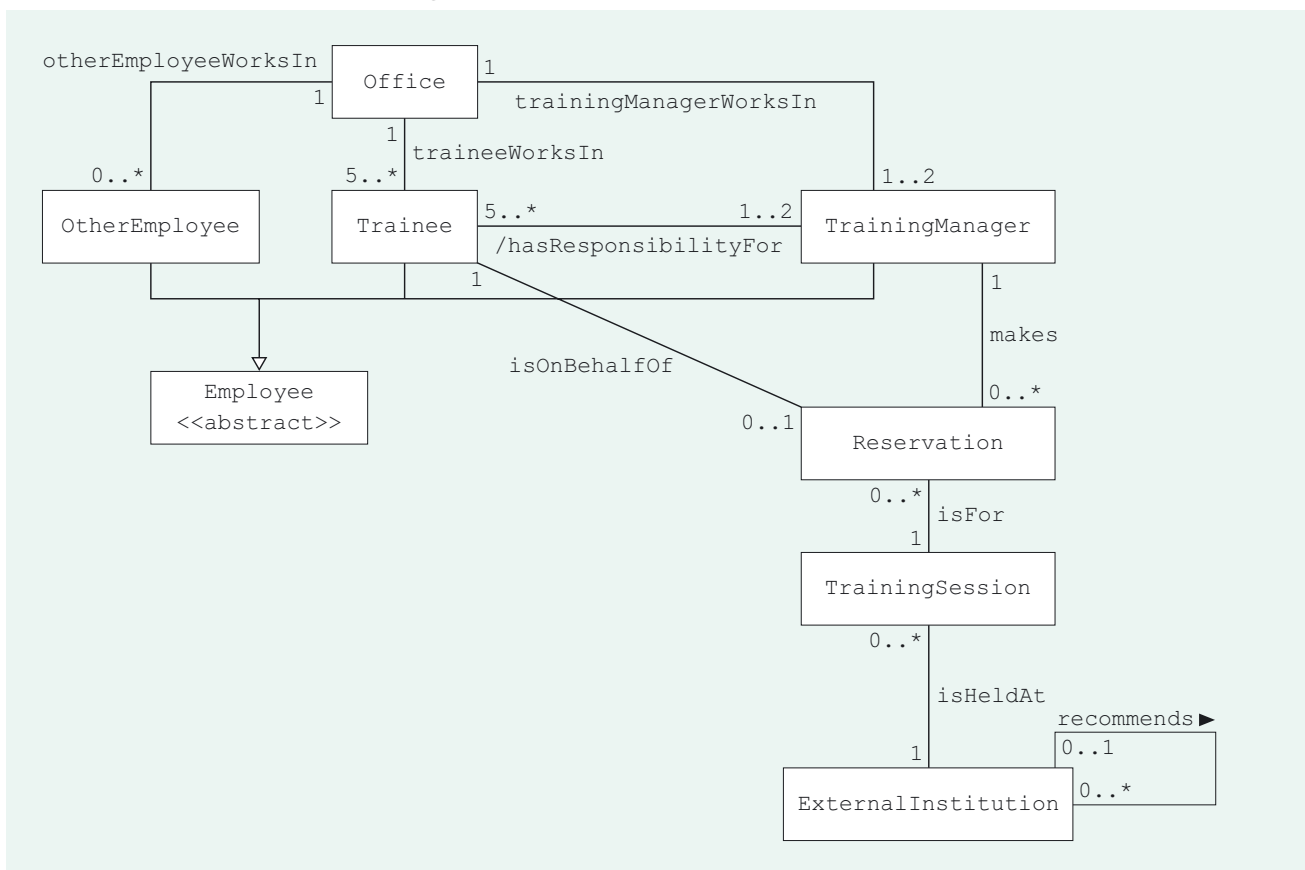


Figure 16 A class diagram for the Training System

Class descriptions

Class	Office	One of the company's offices
Attributes		
	address	The unique address of the office
Class	Employee	An employee of the company Abstract: generalises Trainee, TrainingManager and OtherEmployee
Attributes		
	name	The name of the employee
	employeeNo	The unique employee number of the employee

Class	OtherEmployee	An employee at the company: neither a trainee nor a training manager Specialises Employee
	Attributes	
	None	
Class	Trainee	A trainee at the company Specialises Employee
	Attributes	
	None	
Class	TrainingManager	A training manager at the company Specialises Employee
	Attributes	
	qualifications	The training management qualifications of the training manager
Class	TrainingSession	A training session organised by the company
	Attributes	
	title	The title of the training session
	date	The future date of the training session
Class	ExternalInstitution	An external institution
	Attributes	
	name	The name of the external institution
	address	The address of the external institution
	equipment	The specialist equipment available at the external institution
Class	Reservation	A reservation for a training session
	Attributes	
	None	

Invariants

- 1 For each Reservation object, the Trainee and TrainingManager objects to which it is linked are each linked to the same Office object.
- 2 If a Trainee object and a TrainingManager object are linked to the same Office object, then they are linked to each other via hasResponsibilityFor, and vice versa.
- 3 No ExternalInstitution object may be linked to itself via recommends.

SAQ 9

The class diagram in Figure 16 marks the association `hasResponsibilityFor` as derived. Explain which of the invariants the derivation of `hasResponsibilityFor` is based on.

ANSWER.....

The derivation is based on invariant 2.

Given any `TrainingManager` object, the `Office` object it is linked to via `trainingManagerWorksIn` is also linked via `traineeWorksIn` to `Trainee` objects. Invariant 2 guarantees that these `Trainee` objects are precisely those which are linked to the `TrainingManager` object via `hasResponsibilityFor`. Thus all links of the `hasResponsibilityFor` association can be deduced in this way using links of the `trainingManagerWorksIn` and `traineeWorksIn` associations.

We complete this section by commenting on some aspects of the given solution for Exercise 16.

- ▶ Each of the classes in the diagram, or equivalently named classes, is necessary. ('Company' and 'training department' are rejected as being outside the scope of the system.)
- ▶ We have modelled employees using four classes: an abstract class, `Employee`, with three concrete subclasses, `Trainee`, `TrainingManager` and `OtherEmployee`. The requirements make it clear that there are employees who are neither trainees nor training managers, and indeed one of the requirements of the system is to provide details of *all* employees working in an office.

Another model might have `Employee` as a concrete class with two concrete subclasses `Trainee` and `TrainingManager`. Other employees would be represented by instances of the class `Employee`.

- ▶ It is important to understand why the class `Reservation` is required. Reservations are clearly significant events, involving three other important entities in the system domain; a reservation is made by a training manager, on behalf of a trainee, for a training session. You have learnt that events involving more than two entities are modelled by a class, which is associated with each of the classes representing the entities involved. As our class diagram illustrates, each `Reservation` object is linked to a `Trainee` object, a `TrainingManager` object and a `TrainingSession` object.

- ▶ All the associations in the solution to Exercise 16 are required to model significant real-world connections between entities represented by objects of our classes, according to the requirements document, with the following exceptions.

A model might omit the derived association `hasResponsibilityFor`, because, although it is mentioned in the requirements document, its significance to the system domain is not made explicit.

Another model might replace the three separate associations involving `Office` by a single association between `Office` and `Employee` – `worksIn`, say. This would have the disadvantage that the constraints on the number of trainees and the number of training managers in each office could no longer be specified using multiplicities: additional invariants would be required.

- The requirements document for the Training System gives clear information about the multiplicity of most associations. For example:

Each reservation is made by a single training manager ...

The titles and dates of ... training sessions ... (including those training sessions for which there are currently no reservations) are also displayed.

For some associations, explicit information is not given. For example, it is not explicitly stated in the requirements document that a trainee may have no reservation; common sense dictated our modelling of this aspect with a multiplicity of 0..1 at the `Reservation` end of `isOnBehalfOf`. Where the requirements document is lacking, since you have no client with whom to clarify, common sense must prevail.

In this section you have independently undertaken the creation of a conceptual model, bringing to bear concepts you learnt about earlier in this unit and in *Unit 3*.

4

Summary

This unit reinforced the material presented in *Unit 3*. The concepts, techniques and guidelines introduced in that unit were applied to the task of producing a conceptual model for the Shopping System. In producing this model, the ideas you learnt about in *Unit 3* were extended. For example, you saw that a class may have an association with itself – a recursive association. You also saw that there is rarely one possible conceptual model; different developers may create different models of the system domain.

In the final section of the unit you created a conceptual model by yourself.

The next unit, *Unit 5*, introduces you to some important goals in software development, which guide the decisions to be taken as development progresses. Then, in *Unit 6*, you will begin to learn about the design process. Having created the conceptual model as a representation of the system domain, the design process begins treating the conceptual model of the system domain as an initial structural model of the system.

LEARNING OUTCOMES

Having completed this unit you should be able to:

- ▶ describe and use each of the key terms in this unit and in *Unit 3* (summarised in their respective glossaries);
- ▶ carry out, with increased confidence, all the tasks listed in the learning outcomes for *Unit 3*;
- ▶ model a relationship between entities of the same kind as a recursive association;
- ▶ produce a conceptual model from a requirements document.

Glossary

recursive association An association where the links connect objects belonging to the same class.

Index

A

abstract entity 12

association 8
 derived 29

attribute 14
 derived 29

C

candidate
 association 16
 class 7, 9

class description 5

class diagram 5

conceptual
 class 5
 model 5, 7, 30

constraint 27
 dynamic 28
 static 28

D

derived
 association 29
 attribute 29

E

event 12–13

F

figure of speech 10

I

initial structural model 5

invariant 5, 23

L

language idiom 10

M

multiplicity 17

N

noun 7

noun phrase 7

O

object diagram 18

organisational unit 12

R

real-world
 constraint 28
 entity 5, 7

recursive association 20

rejection guidelines 9

requirements document 5

role 12

S

synonym 7

system domain 5

T

tangible entities 12

textual analysis 7, 16

